Assignment Report

SFWRENG 3S03 (2026W)

Assignment 1

Author(s)

Om Patel	patelo11@mcmaster.ca	PATELO11
Soutrik Guhathakurta	guhathas@mcmaster.ca	GUHATHAS
Jim Software	js@mcmaster.ca	MACID789
Joan Engineer	je@mcmaster.ca	MACIDABC

GitHub URL

`https://github.com/jondoe/a1repo`

February 21, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster SFWRENG by Dr. Istvan David.

1 Task 1

1.1 (a)

Prompt + generative-AI response (three types of tests). Prompt used (ChatGPT): "List three types of tests you would run for an action-adventure game set in post-apocalyptic McMaster University with up to 8 players (co-op + competitive), releasing on PS5, Xbox Series X, Nintendo Switch, and Sega Genesis (16-bit). Keep it practical and explain each test type briefly."

Response from ChatGPT:

1. **Platform Compatibility / Certification Testing**
   Verify the game behaves correctly on each platform's hardware, OS, controllers, save system, and platform rules. Special focus for Sega Genesis: memory limits, sprite/tile constraints, audio channels, input latency, and ensuring the game still fails gracefully (e.g., sprite overflow handling) rather than crashing.

2. **Multiplayer / Networking & Sync Testing**
   Test 1–8 players across co-op and competitive modes for state synchronization (positions, damage, AI states), latency tolerance, reconnect behavior, matchmaking/lobbies, host migration (if used), and ensuring no desyncs or soft-locks during chaotic encounters.

3. **Core Gameplay Functional + Regression Testing**
   Validate core loops: movement/collision, combat, enemy AI behaviors (zombies/professors/deans/squirrels), quest/progression triggers, inventory, checkpoints, save/load, and UI. Add regression tests to ensure fixes do not break previously working mechanics.

1.2 (b)

Two poor/unclear things in (a), and why they are unhelpful.

- **Too broad to be directly actionable.** The test types do not specify concrete pass/fail criteria (for example, what exact result proves sync is correct, or what graceful failure means on Genesis), so it is difficult to convert them into immediate executable test cases.

- **Genesis is mentioned but not operationalized.** The response correctly notes memory and sprite constraints, but it does not define test setup details (scenes, sprite counts, resource budgets, acceptable failure behavior), making the highest-risk platform guidance too abstract.

1.3 (c)

Two good/helpful things in (a), and why they are helpful.

- **It targets the biggest risk areas for this game.** Cross-platform behavior, multiplayer synchronization, and core gameplay stability are typically where severe and

costly defects appear, so these categories provide a strong high-level strategy.

- **It recognizes Sega Genesis as a special case.** Highlighting 16-bit constraints is useful because it makes clear that test design cannot treat all platforms identically and requires platform-specific validation priorities.

1.4 (d)

Three specific tests, plus why these are preferred.

1. **Genesis worst-case sprite/tile budget scenario.**
   Setup: Execute a stress scene on Sega Genesis with 8 enemies, player combat effects, and UI elements simultaneously visible to push sprite limits.
   Pass criteria: No crash or soft-lock occurs, degradation is predictable (e.g., controlled sprite flicker or effect culling), and controls remain responsive.

2. **8-player desync/disconnect robustness.**
   Setup: Run an 8-player session, trigger a high-intensity combat sequence, force one player to disconnect mid-combat, then reconnect after 30–60 seconds.
   Pass criteria: Ongoing players continue without disruption, and the returning player's position/HP/inventory/objectives match the authoritative state without rubber-banding or broken quest progression.

3. **Save/load state integrity during combat.**
   Setup: During an encounter, save at a checkpoint (or manual save), immediately reload, and inspect the full world state.
   Pass criteria: Player stats, ammo, enemy states, quest flags, and location restore correctly with no duplication/loss and no incorrect progression reset.

These tests are preferred because each one has a clear setup and measurable pass/fail outcome rather than only broad categories. They also directly target the highest-risk areas in this scenario: Genesis hardware constraints, multiplayer instability, and progression/state integrity.

2 Task 2

2.1 Program 1: `findLast`

2.1.1 (a)

Fault + precise fix. Fault: The loop stops at `i > 0`, so index `0` is never checked.
Fix: Change the loop condition to include index `0`: `for (int i = x.length - 1; i >= 0; i-)`.

2.1.2 (b)

Test that does not execute the fault. Use `x = null`, `y = any value`. This throws `NullPointerException` at `x.length`, so the loop condition is not evaluated.

2.1.3 (c)

Test that executes the fault but no error state. Example: `x = [2,3,5], y = 3, expected = 1`. The faulty condition exists, but the method returns at index `1`, so index `0` is not needed and behavior matches the specification.

2.1.4 (d)

Error state but not failure. Example: `x = [9,3,5], y = 2, expected = -1`. The error state occurs when `i` becomes `0` and `i > 0` evaluates false, causing loop exit even though the correct version would still execute one more iteration; output remains `-1`, so there is no failure.

2.1.5 (e)

First error state for the given failing test. Given `x = [2,3,5], y = 2` (expected `0`), the first error state occurs when `i` reaches `0` and `i > 0` evaluates false, so control exits the loop and returns `-1` without checking `x[0]`.

2.2 Program 2: `lastZero`

2.2.1 (a)

Fault + precise fix. Fault: The method returns the first zero because it scans left-to-right and returns immediately.
Fix: Scan from the end: `for (int i = x.length - 1; i >= 0; i-) { if (x[i] == 0) return i; } return -1;`.

2.2.2 (b)

Test that does not execute the fault. Example: `x = [1,2,3], expected = -1`. The faulty `return i;` inside the `if` is never executed because no zero exists.

2.2.3 (c)

Test that executes the fault but no error state. Example: `x = [1,2,0], expected = 2`. The method returns at index `2`, which is genuinely the last zero, so no incorrect state is reached.

2.2.4 (d)

Error state but not failure. Example: `x = [1,0,2], expected = 1`. At `i = 1`, control returns immediately without checking later indices, which is a control-state error for a `find last` computation; because no later zero exists, the returned value is still correct.

2.2.5 (e)

First error state for the given failing test. Given `x = [0,1,0]` (expected `2`), the first error state is at `i = 0` when `x[0] == 0` causes immediate `return 0`, committing before checking later indices where another zero exists.

2.3 Program 3: `countPositive`

2.3.1 (a)

Fault + precise fix. Fault: The condition `x[i] >= 0` counts zero as positive.
Fix: Use strict positivity: `if (x[i] > 0) { count++; }`.

2.3.2 (b)

Test that does not execute the fault. Example: `x = [], expected = 0`. The loop body does not run, so `x[i] >= 0` is never executed.

2.3.3 (c)

Test that executes the fault but no error state. Example: `x = [-4,2,5], expected = 2`. No zeros appear, so `>= 0` behaves the same as `> 0` for all elements in this test.

2.3.4 (d)

Error state but not failure. Not possible. If the fault is activated (a `0` appears), `count` increments when it should not, which changes the returned value and therefore causes a failure.

2.3.5 (e)

First error state for the given failing test. Given `x = [-4,2,0,2]` (expected `2`), the first error state occurs at index `i = 2`: `x[i] >= 0` is true for `0`, so `count` becomes `2` when it should still be `1`.

2.4 Program 4: `oddOrPos`

2.4.1 (a)

Fault + precise fix. Fault: The check `x[i] % 2 == 1` fails for negative odd numbers in Java (e.g., `-3 % 2 == -1`).
Fix: Use `x[i] % 2 != 0 || x[i] > 0`.

2.4.2 (b)

Test that does not execute the fault. Example: `x = [], expected = 0`. The loop body does not execute, so the faulty odd-check expression is never evaluated.

2.4.3 (c)

Test that executes the fault but no error state. Example: `x = [-2,0,1,4], expected = 2`. No negative odd values appear, so behavior stays consistent with the specification.

2.4.4 (d)

Error state but not failure. Not possible. When a negative odd value appears, the method fails to count it, reducing `count`; that incorrect internal state directly propagates to the returned value.

2.4.5 (e)

First error state for the given failing test. Given `x = [-3,-2,0,1,4]` (expected `3`), the first error state occurs at `i = 0`: `-3` should be counted as odd, but `-3 % 2 == 1` is false, so `count` remains `0` when it should be `1`.

3 Task 3: Testing parts of large systems

3.1 (a)

The agent's behaviour can be decoupled using **Dependency Injection** and **Test Doubles** (specifically, stubs/fakes) behind the provided `CatanAgent` interface.

The core engine should solely rely on:

- game rules/state transitions, and
- the contract `CatanAgent` (inputs/outputs)

This allows testing the core module with deterministic, pre-scripted agent choices, so tests focus on legality checks and rule execution rather than RL quality. It also makes later replacement easy: random, heuristic, RL, or human-network agents can all plug in without changing core logic.

3.2 (b)

A minimal implementation of the design is implemented in `src/catan`:

- `CatanAgent.java`: given interface (core dependency boundary).
- `AgentPolicy.java`: behavior strategy abstraction used for composition.
- `PolicyBackedCatanAgent.java`: adapter that implements `CatanAgent` by delegating to an injected policy.
- `ScriptedPolicy.java`: deterministic test double with queued moves.
- `CatanCoreEngine.java`: core loop; validates move legality and applies a safe fallback (`PASS`) if illegal.

- `GameState.java`, `Move.java`, enums: minimal domain model.
- `Demo.java`: runs four agents to show separation in practice.

Key point: the engine never knows whether a move came from a stub, heuristic, or RL policy. That behavior is injected.

3.3 (c)

A common solution from a developer unfamiliar with these techniques is to place decision logic directly inside the engine, e.g., `if(agentType == RL) ... else ...`, or to hardcode random/heuristic behavior in core classes.

Problems with that approach and benefits of mine:

- **High coupling**: rule code and behavior code become tangled. In the current design, the core only sees `CatanAgent`.
- **Poor testability**: non-deterministic behavior makes core tests flaky. Scripted policies make tests reproducible.
- **Open/Closed violation**: every new agent type forces edits to core logic. With injection, new policies are added without touching engine code.
- **Higher maintenance risk**: bugs in learning code can destabilize rule validation. Separation keeps failures localized and easier to debug.

3.4 (d)

I would communicate to:

- technical peers (design session + code walkthrough),
- tech lead/manager (delivery and risk argument), and
- QA/testing members (how deterministic doubles improve test quality).

How and what I would communicate:

- short architecture note (problem, decision, tradeoffs, examples),
- live demo with the implemented files showing swappable policies,
- test strategy showing deterministic agent stubs for core-rule tests,
- team conventions: "core depends on interfaces, behavior injected".

Expectations:

- shared vocabulary (DI, test double, strategy),
- more stable and faster tests,
- easier onboarding and safer extension to RL agents.

With respect to the CEAB engineering practice: this reflects disciplined design, verification mindset, communication, professionalism, and continuous process improvement. It applies software engineering knowledge to reduce risk while improving quality and maintainability.

4 Task 4: Test-Driven Development(TDD)

4.1 (a)

The following small initial unit test suite for `divide()` can be used to verify basic functionality and sign semantics:

- `testDividePositiveNumbers`: verifies normal behaviour (`5.0 / 2.0 = 2.5`)
- `testDivideNegativeByPositive`: verifies sign semantics (`-9.0 / 3.0 = -3.0`)

These tests are implemented in `src/calculator/CalculatorTddTest.java`. They define the first iteration's target behaviour before implementing full edge-case handling.

4.2 (b)

The basic functionality has been implemented in `src/calculator/Calculator.java` until the tests passed:

- method `divide(double numerator, double denominator)`
- returns `numerator / denominator` for valid normal inputs

The tests were then executed from the command line and were confirmed to pass.

4.3 (c)

After iteration 1, the functionality is not complete. The first tests only validate nominal arithmetic behaviour and one sign case. Missing behaviour becomes obvious when compared with code one would write non-TDD in one pass, where developers often immediately think about exceptional cases.

Important missing behaviours:

- division by zero (`0.0` and `-0.0`)
- invalid floating input such as `NaN`
- explicit and documented error semantics (which exception and why)

TDD made this visible quickly: once minimal tests pass, the next step is to identify what behaviour is still unspecified. If a behaviour is not in tests, it is effectively not part of the contract yet.

4.4 (d)

The test suite was then expanded by adding a second TDD iteration focused on edge cases:

- `testDivideByZeroThrows`: expects `ArithmeticException` for denominator `0.0`
- `testDivideNegativeZeroThrows`: expects `ArithmeticException` for denominator `-0.0`
- `testDivideNaNThrows`: expects `IllegalArgumentException` if an argument is `NaN`

These tests are also in `src/calculator/CalculatorTddTest.java`. They extend the contract from "works on normal numbers" to "fails safely on invalid input."

4.5 (e)

The `divide()` method was then updated to satisfy the expanded suite:

- validate inputs and reject `NaN` with `IllegalArgumentException`
- reject zero denominator (including `-0.0`) with `ArithmeticException`
- keep normal arithmetic behaviour unchanged

This final behaviour is implemented in `src/calculator/Calculator.java`.

4.6 (f)

I would use TDD in contexts where correctness and regression safety are important: utility/-domain logic, business rules, parsers, and modules with clear input/output contracts.

I would use it less strictly for heavy UI prototyping or exploratory tasks where requirements change rapidly by design.

Element I liked most:

- it forces explicit behaviour contracts and prevents overbuilding

Hardest element to manage:

- selecting the right test granularity and not overfitting implementation details while still covering edge cases

Overall, this exercise showed the TDD cycle clearly across two iterations: start with minimal behaviour, then expand the contract with edge cases and update implementation accordingly.